

第五次作业反馈

魏远毅

2024 年 10 月 23 日

1 习题 3.13

1.1 题干

尝试将下列递归函数改写为非递归形式：

```
1 void func(int &s){
2     int x;
3     scanf("%d", &x);
4     if(x == 0){s=0;return;}
5     func(s);
6     s+=x;
7     printf("s = %d\n", s);
8 }
```

1.2 解答

先观察递归函数的结构：1. 单递归；2. 主操作”s+=x”在递归调用操作”func(s)”后面；因此，所有对 s 的操作都是在递归到达底部后才进行的，即”x==0”后进行；这个递归结构类似于栈，即满足后入后出的特点。因此，我们考虑采用栈来存储 x 的值，在存储完成后 pop 出栈顶元素，并将其与 s 相加。具体伪代码如下：

```
1 void func(int &s){
2     int x, s = 0;
3     InitStatk_sq(st);
4     while(true){
5         scanf("%d", &x);
6         Push_sq(st, x); //入栈
7         if(x == 0){break;}
8     }
9     while(!IsEmpty_sq(st)){
10        s+= Pop_sq(st); //出栈
11        printf("s=%d\n", s);
12    }
13 }
```

1.3 变体

在将递归函数转非递归函数时，如果是单递归，只需要观察你需要转换的操作在递归调用操作的前面还是后面，在前面通常考虑使用队列的结构转换（也可以直接转换），在后面通常考虑使用栈的结构转换。

如果将题目中的伪代码改成这样：

```
1 int s = 0;
2 void func(int s){
3     int x;
4     scanf("%d", &x);
5     if(x == 0){return;}
6     s+=x;
7     printf("s=%d\n", s);
8     func(s);
9 }
```

对于这样先入先出的递归结构，我们就可以直接用循环将其转换为非递归函数。

2 习题 3.15：写一个递归算法，对不带头结点的单链表进行逆序，要求输入参数是单链表的头指针，返回逆序之后的头指针。

常规思路就是将长度为 n 的链表拆分为 $1+(n-1)$ 两部分，对后面这部分进行逆序，然后将这两部分交换位置即可。伪代码如下：

```
1 Link_node inverse(Link_node *head){
2     Link_node *new_head;
3     if(head->next == NULL) return head; //递归终止条件
4     else{
5         new_head = inverse(head->next);
6         head->next->next = head;
7         head->next = NULL;
8         return new_head;
9     }
10 }
```

补充说明：部分同学在对 $(n-1)$ 部分链表逆序后，通过遍历操作来寻找此时 $(n-1)$ 部分的尾节点。但其实此时该尾节点就是原来链表中的第二个节点，因此我们可以直接用 `head->next` 找到尾节点。

3 习题 4.2：已知模式串 $P = \text{"abcaabbcb"}$ ，求其改进后的 next 数组。

对于该问题，有两种解决思路：1. 按照给定算法进行计算；2. 按照 next 数组的意义进行计算。

下面介绍第二种思路：

“ $\text{next}[j]=k$ ”表示在模式串 P 中，满足： $P[0:k-1]=P[j-k:j-1]$

因此，对于每个 j ，往前观察即可得到 next 数组，例如： $j=6$ ，有 $P[0:1]=P[4:5]$ ，则 $\text{next}[6]=2$

next_new 则是在“ $\text{next}[j]=k$ ”的基础上，考虑条件“ $P[k]=P[j]$ ”的情况，如果该情况成立，则 $\text{next_new}[j]=\text{next}[k]$ 计算结果如下：

	a	b	c	a	a	b	b	c	a	b
next[j]	-1	0	0	0	1	1	2	0	0	1
next_new[j]	-1	0	0	-1	1	0	2	0	-1	0